



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Propuesta del Proyecto DataQuest: Validador de Normalización de Bases de Datos Relacionales

Curso: Base de Datos II

Docente: Ing. Patrick Jose Cuadros Quiroga

Integrantes:

Dongo Palza, Manuel Andre (2023076802)

Perez Peralta, Fabrizio Salvador Elias (2023077476)

**Tacna - Perú
2026**

CONTROL DE VERSIONES

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	MADP	Pendiente de revisión docente	Pendiente	03/07/2026	Versión base de propuesta del proyecto DataQuest.

Contenido

CONTROL DE VERSIONES	2
RESUMEN EJECUTIVO	4
Resultados esperados principales	4
PROPUESTA NARRATIVA	5
1. PLANTEAMIENTO DEL PROBLEMA	7
Causas principales	7
Efectos del problema	7
2. JUSTIFICACIÓN DEL PROYECTO	8
Justificación técnica	8
Justificación académica	8
Justificación económica	8
Justificación social	8
Justificación ambiental	8
Justificación metodológica	8
3. OBJETIVO GENERAL	8
Objetivos específicos	8
4. BENEFICIOS	9
Beneficios tangibles	9
Beneficios intangibles	9
5. ALCANCE	10
Alcance funcional	10
Alcance técnico	10
Fuera del alcance	10
6. REQUERIMIENTOS DEL SISTEMA	11
Requerimientos no funcionales	12
7. RESTRICCIONES	12
8. SUPUESTOS	12
9. RESULTADOS ESPERADOS	13
10. METODOLOGÍA DE IMPLEMENTACIÓN	13
Arquitectura propuesta	14
11. ACTORES CLAVES	14

12. PAPEL Y RESPONSABILIDADES DEL PERSONAL.....	15
13. PLAN DE MONITOREO Y EVALUACIÓN	15
14. CRONOGRAMA DEL PROYECTO	15
15. HITOS DE ENTREGABLES	16
PRESUPUESTO	17
16. PLANTEAMIENTO DE APLICACIÓN DEL PRESUPUESTO	17
17. PRESUPUESTO	17
Escenarios de presupuesto.....	18
18. ANÁLISIS DE FACTIBILIDAD	18
19. EVALUACIÓN FINANCIERA.....	18
Supuestos de evaluación.....	19
20. Resumen técnico ejecutivo.....	19
20.1 Valor diferencial de DataQuest.....	19
20.2 Principios rectores de la propuesta	20
21. Diagnóstico profundo del proyecto DataQuest.....	20
21.1 Inventario técnico real del repositorio	20
21.2 Hallazgos QA relevantes.....	20
21.3 Decisiones técnicas que deben mantenerse	21
22. Análisis estratégico y propuesta de valor ampliada.....	21
22.1 Población objetivo segmentada.....	21
22.2 Propuesta de valor resumida	21
22.3 Alineamiento con objetivos académicos	21
23. EDT/WBS, backlog e historias de usuario.....	22
23.1 Backlog de producto priorizado	22
23.2 Historias de usuario detalladas con criterios de aceptación.....	22
24. Arquitectura técnica, ciclo de datos, API y seguridad.....	23
24.1 Arquitectura lógica enriquecida	23
24.2 Especificación ampliada del endpoint principal.....	24
24.3 Seguridad y privacidad.....	24
24.4 Plan DevOps e infraestructura	24
25. Plan QA extendido y criterios de aceptación	25
25.1 Objetivo del plan QA	25
25.2 Matriz de pruebas funcionales principales.....	25
25.3 Criterios de aceptación globales	25
26. Gobierno, monitoreo, comunicación y adopción.....	26
26.1 Matriz RACI extendida	26
26.2 Plan de monitoreo y evaluación enriquecido	27

26.3 Plan de comunicación	27
26.4 Plan de capacitación y adopción	27
27. Presupuesto enriquecido y evaluación financiera académica	28
27.1 Supuestos presupuestales	28
27.2 Presupuesto por escenarios.....	28
27.3 Retorno académico esperado	28
27.4 Evaluación beneficio/costo referencial	29
28. Riesgos, mitigaciones y trazabilidad ampliada	29
28.1 Matriz de riesgos extendida	29
28.2 Trazabilidad problema-objetivo-requerimiento-prueba.....	29

RESUMEN EJECUTIVO

Campo	Descripción
Nombre del proyecto propuesto	DataQuest: Validador de Normalización de Bases de Datos Relacionales.
Propósito del proyecto	Implementar una plataforma académica y técnica para analizar esquemas relacionales, validar reglas 1FN, 2FN y 3FN, generar diagnósticos, sugerencias y evidencias que apoyen el aprendizaje del diseño lógico de bases de datos.
Población objetivo	Estudiantes de bases de datos, docentes, desarrolladores principiantes, evaluadores académicos y usuarios que requieren revisar esquemas SQL con fines educativos.
Monto de inversión referencial	S/ 13,500.00 como presupuesto recomendado académico. Se presenta desglose mínimo, recomendado y extendido.
Duración del proyecto	4 meses, organizado en sprints incrementales de análisis, motor, interfaz, API, persistencia, pruebas y documentación.

Resultados esperados principales

- Plataforma web operativa mediante Streamlit para ingresar esquemas SQL y revisar resultados de normalización.
- Motor Python capaz de analizar sentencias CREATE TABLE, extraer tablas, columnas, claves primarias y claves foráneas.
- Validación de reglas de normalización 1FN, 2FN y 3FN con diagnóstico comprensible para estudiantes.
- API FastAPI disponible para consumo externo mediante el endpoint POST /api/normalizacion.
- Persistencia o historial en Supabase/PostgreSQL cuando el entorno esté configurado con las variables de ambiente correspondientes.
- Extensión de Visual Studio Code y CLI como canales técnicos de integración, según lo evidenciado en el repositorio.
- Plan de QA con pruebas unitarias, integración, interfaz, BDD, API, errores controlados y regresión.
- Documentación técnica y académica alineada a FD03/SRS, FD05/Informe Final y FD06/Propuesta de Proyecto.

Resumen de viabilidad: La propuesta se considera técnica, académica y económicamente viable para un entorno universitario. El proyecto utiliza tecnologías accesibles, separa la lógica del motor de normalización de la interfaz, expone una API para integración externa y permite documentar evidencias de aprendizaje. Su valor principal no es reemplazar al docente, sino reducir la revisión preliminar manual y mejorar la comprensión del estudiante.

PROPUESTA NARRATIVA

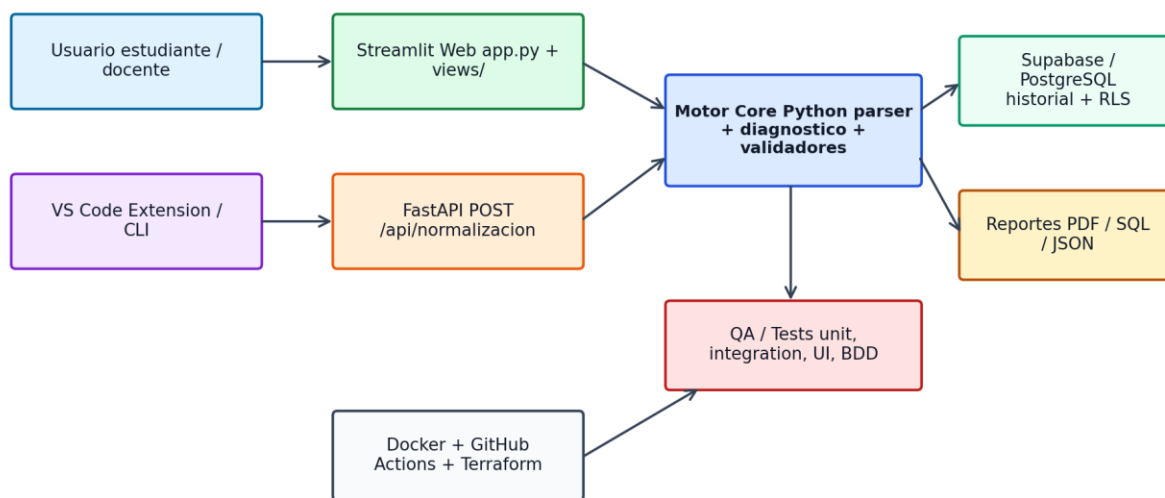
DataQuest se propone como una plataforma educativa y técnica para mejorar el proceso de aprendizaje y validación de esquemas de bases de datos relacionales. Su enfoque parte de una dificultad frecuente en cursos de base de datos: comprender y aplicar correctamente las reglas de normalización hasta la Tercera Forma Normal. Aunque la teoría de normalización suele explicarse mediante definiciones y ejemplos aislados, los estudiantes enfrentan mayores dificultades cuando deben analizar esquemas SQL completos, justificar claves, reconocer dependencias funcionales y proponer correcciones defendibles.

La propuesta plantea transformar esta revisión en un proceso guiado, repetible y verificable. El usuario podrá ingresar sentencias CREATE TABLE desde la interfaz web o desde otros canales técnicos, como API, CLI o extensión de Visual Studio Code. El sistema analizará el esquema, identificará estructuras relevantes, evaluará posibles incumplimientos de 1FN, 2FN y 3FN, y mostrará un diagnóstico orientado al aprendizaje. Este diagnóstico no se limita a marcar errores: también debe explicar el motivo, indicar el nivel afectado y proponer sugerencias de mejora.

Desde el punto de vista técnico, DataQuest integra un motor core en Python con módulos separados para parsing, diagnóstico, validadores y generación de sugerencias. La interfaz Streamlit facilita el uso académico, mientras que FastAPI permite exponer el motor como servicio HTTP/JSON para integraciones externas. Supabase/PostgreSQL aporta persistencia e historial cuando la configuración del entorno esté disponible, y la extensión de VS Code permite acercar el análisis al editor de código del estudiante.

La propuesta también reconoce límites importantes. DataQuest no sustituye una evaluación formal del docente ni garantiza una interpretación perfecta de toda dependencia funcional si el usuario no proporciona información suficiente. En casos complejos, la normalización puede requerir criterio experto y definición explícita de dependencias funcionales. Por ello, el sistema se presenta como herramienta académica de apoyo, diagnóstico preliminar y retroalimentación guiada, no como certificador absoluto de modelos relacionales.

Con esta propuesta, DataQuest busca generar valor académico mediante una solución concreta, defendible y medible: reducir el tiempo de revisión preliminar, mejorar la trazabilidad de las correcciones, apoyar al estudiante en la sustentación de sus diseños y promover buenas prácticas de modelado relacional.



Fuente: elaboración propia basada en el análisis del repositorio DataQuest, FD03/SRS y FD05.

Figura 1. Arquitectura general propuesta de DataQuest. Fuente: elaboración propia.

1. PLANTEAMIENTO DEL PROBLEMA

Los estudiantes y desarrolladores principiantes presentan dificultades para identificar, justificar y corregir incumplimientos de normalización en esquemas de bases de datos relacionales. Esta dificultad se evidencia especialmente cuando deben analizar reglas de Primera Forma Normal, Segunda Forma Normal y Tercera Forma Normal, interpretar claves primarias simples o compuestas, reconocer dependencias funcionales, separar atributos multivaluados y prevenir anomalías de inserción, actualización y eliminación.

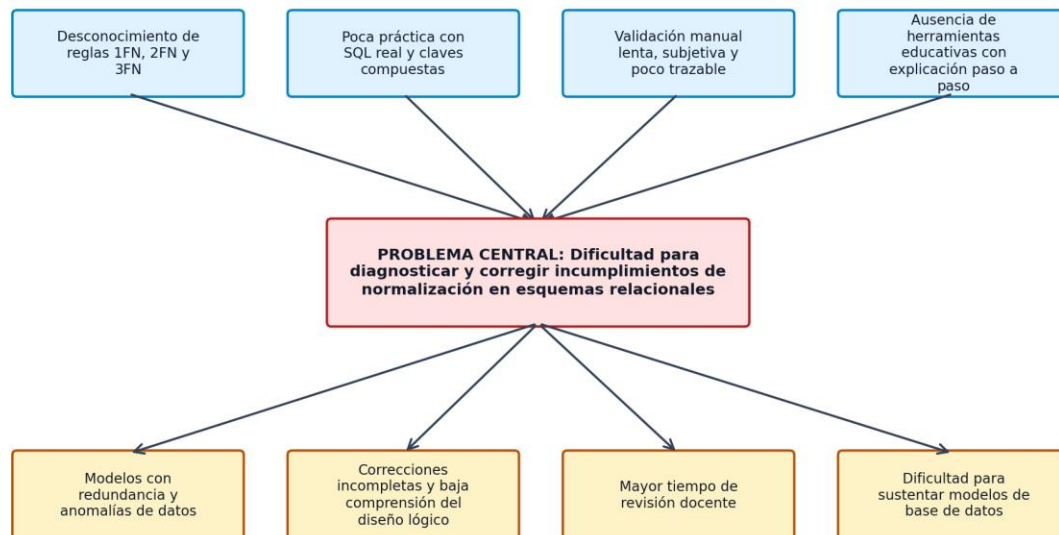
La revisión manual puede convertirse en una actividad lenta, subjetiva y poco trazable. Un mismo esquema puede ser interpretado de forma distinta si no se dispone de criterios claros o de evidencia técnica. Además, cuando el estudiante solo recibe una observación final sin explicación paso a paso, la corrección puede volverse mecánica y no necesariamente mejora la comprensión del diseño lógico.

Causas principales

- Desconocimiento práctico de las reglas 1FN, 2FN y 3FN.
- Poca experiencia con sentencias SQL reales y claves compuestas.
- Dificultad para detectar dependencias parciales y transitivas.
- Ausencia de retroalimentación automática y trazable.
- Uso frecuente de esquemas con redundancia, atributos multivaluados y relaciones ambiguas.
- Falta de reportes técnicos que permitan sustentar la corrección del modelo.

Efectos del problema

- Modelos relacionales con redundancia y anomalías de datos.
- Correcciones incompletas o justificadas solo de manera verbal.
- Mayor tiempo de revisión por parte del docente.
- Baja comprensión del diseño lógico y de las dependencias funcionales.
- Dificultad para defender el modelo durante una exposición o entrega académica.
- Repetición de errores en proyectos posteriores de base de datos.



Fuente: elaboración propia basada en el análisis del proyecto DataQuest.

Figura 2. Árbol de problemas de DataQuest. Fuente: elaboración propia.

2. JUSTIFICACIÓN DEL PROYECTO

Justificación técnica

DataQuest se justifica técnicamente porque utiliza una arquitectura modular basada en Python, Streamlit, FastAPI y Supabase/PostgreSQL. El repositorio evidencia separación entre core, controllers, views, db, visualización y tests, lo cual facilita mantenimiento, pruebas y evolución. La existencia de Dockerfile, workflows de GitHub Actions, Terraform, CLI y extensión de VS Code fortalece la propuesta técnica al permitir despliegue, automatización e integración con otros entornos.

Justificación académica

El sistema atiende una necesidad recurrente en cursos de bases de datos: pasar de la teoría de normalización a la aplicación práctica sobre esquemas SQL. La herramienta permite experimentar con errores reales, recibir diagnóstico y entender por qué una tabla incumple una forma normal.

Justificación económica

La propuesta es económicamente viable en un entorno académico porque emplea tecnologías libres o de bajo costo. El presupuesto se concentra en desarrollo, documentación, pruebas, diagramación, despliegue controlado y soporte de sustentación.

Justificación social

Al mejorar la formación de estudiantes de ingeniería de sistemas, DataQuest contribuye a elevar la calidad de los proyectos académicos de bases de datos y reduce la brecha entre conocimiento teórico y práctica técnica.

Justificación ambiental

La propuesta favorece evidencias digitales, reportes electrónicos y revisión automatizada, reduciendo la necesidad de imprimir guías, borradores, observaciones y correcciones manuales.

Justificación metodológica

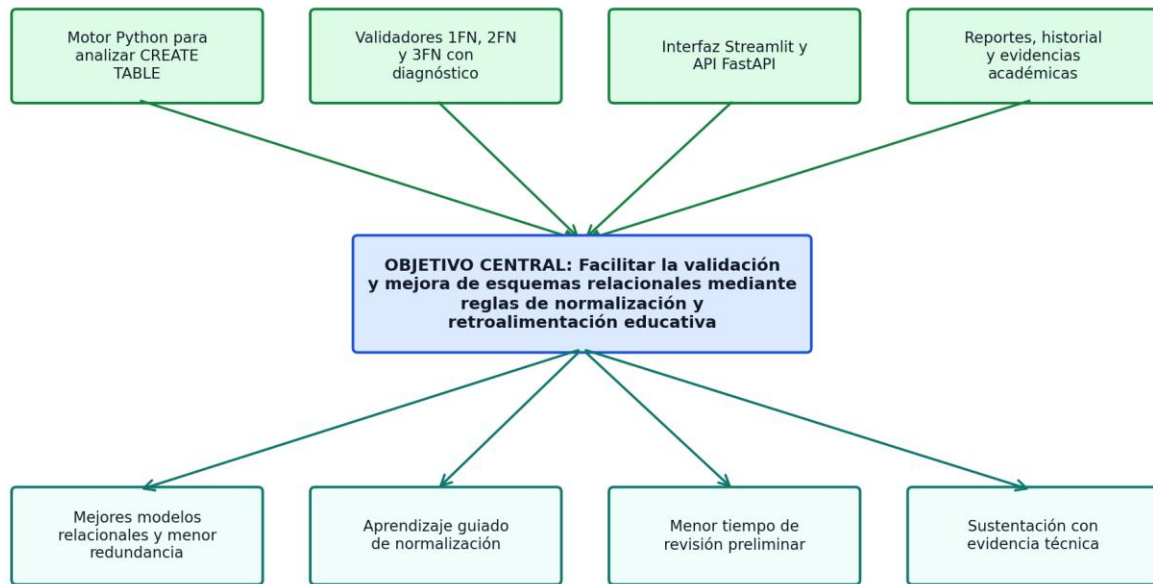
El sistema permite que la validación sea estructurada, repetible y trazable. Cada revisión puede asociarse a un diagnóstico, un nivel de normalización, una sugerencia y una evidencia de prueba.

3. OBJETIVO GENERAL

Implementar una plataforma académica denominada DataQuest, orientada a validar y mejorar esquemas de bases de datos relacionales mediante reglas de normalización 1FN, 2FN y 3FN, integrando una interfaz web, un motor lógico en Python, una API FastAPI y mecanismos de persistencia o reporte, con el fin de generar diagnósticos, sugerencias y evidencias que fortalezcan el aprendizaje del diseño lógico de bases de datos.

Objetivos específicos

- OE1: Analizar sentencias SQL CREATE TABLE ingresadas por el usuario.
- OE2: Identificar estructuras que incumplan Primera Forma Normal.
- OE3: Detectar posibles dependencias parciales asociadas a Segunda Forma Normal.
- OE4: Detectar posibles dependencias transitivas asociadas a Tercera Forma Normal.
- OE5: Generar diagnósticos comprensibles y orientados al aprendizaje.
- OE6: Proponer sugerencias de corrección de esquemas relacionales.
- OE7: Exponer el motor mediante una API FastAPI.
- OE8: Registrar historial o evidencia de validaciones cuando la persistencia esté configurada.
- OE9: Validar el sistema mediante pruebas unitarias, funcionales, API y checklist QA.
- OE10: Documentar la solución para sustentación académica y continuidad técnica.



Fuente: elaboración propia basada en el análisis del proyecto DataQuest.

Figura 3. Árbol de objetivos de DataQuest. Fuente: elaboración propia.

4. BENEFICIOS

Los beneficios de DataQuest se clasifican en tangibles e intangibles. Los primeros pueden relacionarse con reducción de tiempo, reutilización de la herramienta y centralización de evidencias; los segundos se vinculan con mejora del aprendizaje, seguridad del estudiante al sustentar y fortalecimiento de buenas prácticas de diseño relacional.

Concepto	Situación actual	Beneficio con DataQuest	Impacto esperado
Tiempo de revisión manual	Corrección manual caso por caso, con observaciones dispersas.	Diagnóstico preliminar automatizado y reutilizable.	Reducción estimada del 40% al 60% del tiempo de revisión inicial en prácticas académicas.
Evidencia académica	Sustentación basada en explicación verbal o capturas no estandarizadas.	Reporte técnico con nivel de normalización, violaciones y sugerencias.	Mayor trazabilidad de decisiones y correcciones.
Aprendizaje de normalización	El estudiante memoriza reglas, pero no siempre las aplica correctamente.	Retroalimentación por 1FN, 2FN y 3FN aplicada sobre SQL real.	Mejora en comprensión de dependencias y anomalías.
Reutilización	Cada práctica se corrige desde cero.	La plataforma puede usarse en múltiples prácticas y cursos.	Mayor eficiencia docente y estudiantil.
Calidad de proyectos	Modelos con redundancia o claves ambiguas.	Sugerencias de separación y normalización.	Incremento de calidad técnica en entregables.

Beneficios tangibles

- Reducción del tiempo de revisión manual de esquemas.
- Generación de evidencias digitales y reportes reutilizables.
- Estandarización de criterios de diagnóstico preliminar.
- Disminución de correcciones repetitivas en prácticas académicas.
- Reutilización del sistema en cursos de bases de datos, ingeniería de software y proyectos integradores.

Beneficios intangibles

- Mayor confianza del estudiante al justificar sus decisiones de diseño.
- Fortalecimiento del pensamiento lógico y relacional.
- Mejor comprensión de dependencias funcionales.

- Promoción de buenas prácticas de modelado desde etapas tempranas.
- Mayor claridad en la sustentación académica de proyectos.

5. ALCANCE

Alcance funcional

- Ingreso de sentencias SQL CREATE TABLE por texto, archivo o canal técnico.
- Análisis de tablas, columnas, claves primarias y claves foráneas.
- Validación de 1FN, 2FN y 3FN.
- Generación de diagnóstico, sugerencias y mejoras opcionales.
- Visualización de resultados mediante Streamlit.
- Consumo del motor mediante FastAPI.
- Uso mediante CLI y extensión de Visual Studio Code según evidencia del repositorio.
- Historial en Supabase/PostgreSQL si las variables de entorno y tablas están configuradas.
- Pruebas y evidencias para sustentación académica.

Alcance técnico

- Python 3.10+ como lenguaje principal.
- Streamlit como interfaz web.
- FastAPI como capa de integración HTTP/JSON.
- Supabase/PostgreSQL para autenticación, presencia e historial cuando esté configurado.
- Docker para empaquetado y despliegue controlado.
- GitHub Actions para pruebas, seguridad y despliegue.
- Terraform para infraestructura como código cuando se use entorno cloud.
- Postman/curl para validación de endpoint.
- Pytest, Playwright y BDD como base de pruebas, con necesidad de fortalecer cobertura.

Fuera del alcance

- No reemplaza una evaluación formal del docente.
- No garantiza detección perfecta de toda dependencia funcional si el usuario no ingresa información suficiente.
- No sustituye el análisis conceptual completo ni el diseño entidad-relación previo.
- No corrige automáticamente todos los modelos complejos.
- No debe usarse como decisión única en contextos productivos críticos.
- No debe almacenar ni exponer credenciales sensibles.

6. REQUERIMIENTOS DEL SISTEMA

ID	Descripción	Prioridad	Módulo	Estado	Criterio de aceptación
RF-01	El sistema debe permitir ingresar sentencias SQL CREATE TABLE mediante texto pegado o archivo.	Alta	Streamlit / Parser	Implementado / parcial	El usuario ingresa SQL y el sistema inicia análisis sin error.
RF-02	El sistema debe extraer tablas, columnas, claves primarias y claves foráneas desde el SQL.	Alta	core/parser.py	Implementado	El parser retorna estructura con tablas, columnas, pks y fks.
RF-03	El sistema debe validar Primera Forma Normal detectando ausencia de PK y atributos multivaluados o repetitivos.	Alta	validador_1fn.py	Implementado	El diagnóstico identifica violaciones 1FN con mensaje y tabla.
RF-04	El sistema debe validar Segunda Forma Normal a partir de claves compuestas y dependencias funcionales.	Alta	validador_2fn.py	Implementado / heurístico	Detecta dependencias parciales cuando existen FDs o inferencias compatibles.
RF-05	El sistema debe validar Tercera Forma Normal detectando posibles dependencias transitivas.	Alta	validador_3fn.py	Implementado / heurístico	Detecta dependencias transitivas y degrada el nivel alcanzado.
RF-06	El sistema debe inferir dependencias funcionales cuando el usuario no las ingrese.	Media	diagnostico.py	Implementado / heurístico	El motor aplica reglas por patrones de nombres de columnas.
RF-07	El sistema debe generar sugerencias de corrección con impacto, confianza y beneficios.	Alta	corrector.py	Implementado	Cada violación produce sugerencia pedagógica.
RF-08	El sistema debe generar salida JSON mediante CLI.	Alta	core/cli.py	Implementado	El comando retorna JSON con nivel, violaciones y tablas detectadas.
RF-09	El sistema debe exponer API HTTP/JSON para normalización.	Alta	api.py	Implementado	POST /api/normalizacion responde success, nivel y violaciones.
RF-10	El sistema debe manejar SQL vacío o sin tablas válidas con error controlado.	Alta	api.py / cli.py	Implementado	Devuelve mensaje de error sin stacktrace al usuario.
RF-11	El sistema debe presentar resultados en la interfaz web Streamlit.	Alta	views/05_resultados.py	Implementado / parcial	El usuario visualiza reporte posterior al análisis.
RF-12	El sistema debe guardar historial de validaciones por usuario.	Media	db/modelos.sql	Planificado / requiere entorno	La tabla historial_validaciones existe en script; depende de Supabase configurado.
RF-13	El sistema debe aplicar RLS para proteger historial por usuario.	Alta	db/modelos.sql	Planificado / configurado por SQL	Las políticas RLS limitan acceso al user_id.
RF-14	El sistema debe incluir autenticación y perfiles.	Media	controllers/auth_controller.py / db/auth.py	Parcial	Login/registro dependen de Supabase configurado.
RF-15	El sistema debe permitir uso desde Visual Studio Code.	Media	vscode-extension	Implementado técnico	La extensión ejecuta core/cli.py e inyecta reporte al documento.
RF-16	El sistema debe permitir ejecución por Docker.	Media	Dockerfile	Implementado técnico	Se puede construir imagen si dependencias están disponibles.
RF-17	El sistema debe contar con automatización CI para pruebas.	Media	.github/workflows/tests.yml	Implementado técnico / pruebas mejorables	Workflow existe; pruebas reales deben ampliarse.
RF-18	El sistema debe contar con plan QA y casos de prueba.	Alta	tests/	Parcial	Existen carpetas de pruebas, pero algunas son dummy o pass.
RF-19	El sistema debe generar reportes PDF/SQL si el módulo está disponible.	Media	generador_pdf.py / generador_sql.py	Parcial	Debe verificarse integración completa desde UI.
RF-20	El sistema debe documentar instalación, API e integración.	Alta	README / INTEGRACION_API	Implementado	Documentación base disponible.

Requerimientos no funcionales

ID	Categoría	Requerimiento no funcional	Prioridad	Criterio de aceptación
RNF-01	Usabilidad	La interfaz debe explicar resultados con lenguaje comprensible para estudiantes.	Alta	El usuario entiende el nivel y las sugerencias sin revisar código.
RNF-02	Rendimiento	El análisis de esquemas académicos debe responder en menos de 3 segundos en entorno local.	Media	Validación de esquemas pequeños sin bloqueo perceptible.
RNF-03	Seguridad	No deben exponerse credenciales Supabase, tokens ni DATABASE_URL en repositorio.	Crítica	Variables deben estar en .env y .env.example sin secretos.
RNF-04	Mantenibilidad	El código debe mantenerse separado por core, controllers, views, db, visualización y tests.	Alta	Cada responsabilidad tiene carpeta clara.
RNF-05	Portabilidad	Debe ejecutarse en Windows/Linux con Python 3.10+ y dependencias documentadas.	Media	Instalación reproducible mediante requirements.txt.
RNF-06	Interoperabilidad	La API debe responder JSON estándar y documentarse con OpenAPI/FastAPI docs.	Alta	/docs disponible al ejecutar FastAPI.
RNF-07	Trazabilidad	Cada diagnóstico debe vincularse a tabla, regla, violación y sugerencia.	Alta	Reporte muestra nivel, violación, tabla y mensaje.
RNF-08	Calidad académica	El sistema debe diferenciar diagnóstico automático de criterio docente formal.	Alta	Se muestra alcance educativo y limitaciones.
RNF-09	Pruebas	Debe existir suite de pruebas con casos reales y de regresión.	Alta	Pytest ejecuta pruebas no triviales sobre parser y validadores.
RNF-10	Escalabilidad futura	Debe permitir agregar BCNF, 4FN o ingreso manual avanzado de FDs.	Media	Arquitectura core permite nuevos validadores.
RNF-11	Disponibilidad	El modo local debe funcionar aunque Supabase no esté configurado, salvo módulos de historial/auth.	Media	Validación core no depende de conexión externa.
RNF-12	Documentación	README, integración API y manual de ejecución deben estar actualizados.	Alta	Documentos describen instalación y uso.

7. RESTRICCIONES

- La precisión del diagnóstico depende de la calidad del SQL ingresado y de las dependencias funcionales explícitas o inferidas.
- Algunas dependencias funcionales requieren criterio humano o ingreso manual para evitar falsos positivos.
- El motor actual puede utilizar heurísticas, especialmente para 2FN y 3FN cuando no se proporcionan FDs.
- La API requiere entorno Python y dependencias instaladas.
- Supabase requiere conexión a internet si se usa como servicio externo.
- Docker requiere instalación previa y permisos del sistema operativo.
- La extensión de VS Code requiere Node.js, compilación TypeScript y workspace con core/cli.py.
- El sistema no debe exponer claves, tokens ni credenciales.
- El alcance es académico y no certificador formal.

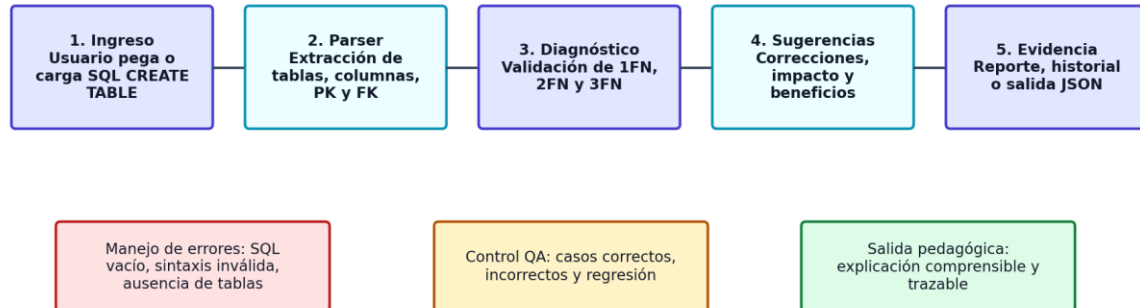
8. SUPUESTOS

- El usuario ingresará SQL válido o razonablemente estructurado.
- El docente permitirá el uso de evidencias digitales como parte de la sustentación.
- El entorno local contará con Python 3.10+ instalado.
- El equipo tendrá acceso al repositorio y a las dependencias del proyecto.

- Las pruebas se realizarán con esquemas correctos, incorrectos y casos límite.
- La documentación se mantendrá actualizada durante el desarrollo.
- El proyecto se ejecutará primero en un entorno académico/local antes de un despliegue público.
- El uso de Supabase se realizará con claves configuradas en variables de entorno.

9. RESULTADOS ESPERADOS

ID	Resultado esperado	Criterio de cumplimiento	Evidencia
RE-01	Plataforma Streamlit funcional	Usuario puede abrir interfaz, ingresar SQL y ejecutar validación.	Captura de pantalla / video demo
RE-02	Motor de validación operativo	Parser y validadores retornan diagnóstico por 1FN/2FN/3FN.	Pruebas unitarias y reporte JSON
RE-03	API FastAPI documentada	Endpoint principal responde y se prueba desde Postman/curl.	Captura de /docs y prueba POST
RE-04	Historial o evidencia persistente	Validaciones pueden guardarse si Supabase está configurado.	Registro en tabla historial validaciones
RE-05	VS Code Extension funcional	Reporte se inyecta en archivo SQL desde el editor.	Captura de extensión o demo
RE-06	Plan QA ejecutable	Casos de prueba definidos con entradas y resultados esperados.	Matriz QA
RE-07	Documentación final	FD06, manual, anexos y checklist listos.	Documento final revisado



Fuente: elaboración propia basada en el análisis del proyecto DataQuest.

Figura 4. Flujo funcional del proceso de validación. Fuente: elaboración propia.

10. METODOLOGÍA DE IMPLEMENTACIÓN

La implementación se plantea mediante un enfoque incremental basado en SCRUM académico. Cada sprint genera un entregable verificable, permitiendo validar el avance de forma continua y reducir riesgos de integración. El proyecto se organiza en 4 meses, con énfasis en análisis, motor core, interfaz, API, persistencia, pruebas, documentación y sustentación.

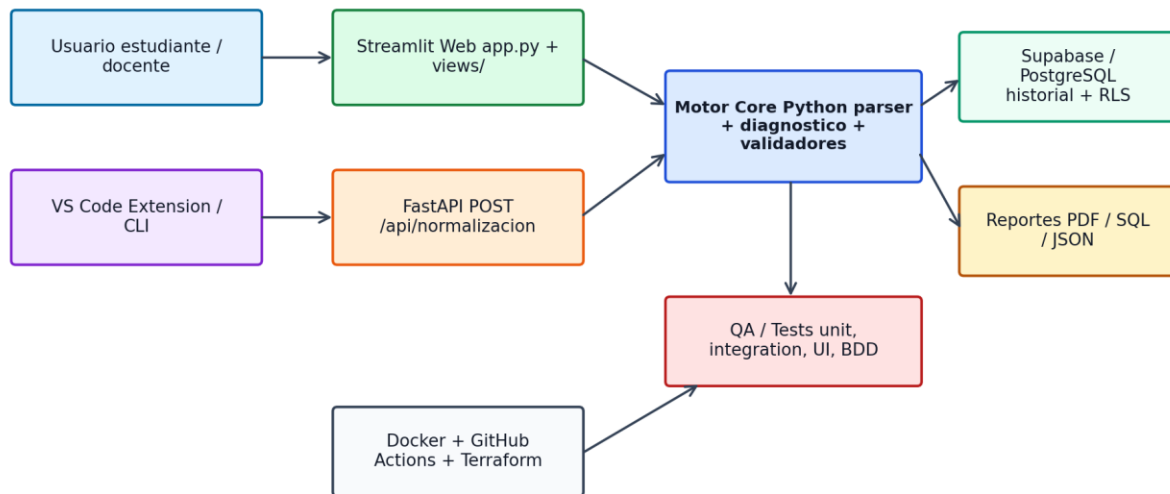
Sprint	Duración	Objetivo	Actividades	Entregable	Criterio de aceptación
Sprint 0	1 semana	Preparación	Revisión de FD03, FD05, repositorio, alcance, backlog y entorno.	Backlog y entorno base	Alcance aprobado.

DataQuest - FD06 / Propuesta de Proyecto

Sprint 1	2 semanas	Análisis y requisitos	Formalizar problema, actores, RF/RNF y criterios de aceptación.	Requerimientos consolidados	RF y RNF revisados.
Sprint 2	2 semanas	Parser SQL	Extraer tablas, columnas, PK y FK desde CREATE TABLE.	Parser funcional	SQL de prueba detecta estructura.
Sprint 3	2 semanas	Validadores	Fortalecer validadores 1FN, 2FN y 3FN.	Motor core	Casos de prueba pasan.
Sprint 4	2 semanas	Interfaz Streamlit	Mejorar entrada, resultados y mensajes pedagógicos.	UI funcional	Usuario ejecuta flujo completo.
Sprint 5	2 semanas	API FastAPI	Validar endpoint, errores y documentación OpenAPI.	API funcional	POST /api/normalizacion probado.
Sprint 6	2 semanas	Persistencia	Configurar Supabase/PostgreSQL, RLS e historial.	BD funcional	Historial por usuario protegido.
Sprint 7	2 semanas	DevOps y QA	Docker, CI, seguridad, pruebas y regresión.	Pipeline y QA	Workflow ejecuta pruebas reales.
Sprint 8	1 semana	Cierre	Documentación, evidencias, sustentación y correcciones.	FD06 final	Entrega lista para revisión.

Arquitectura propuesta

La arquitectura propuesta separa la interfaz de usuario, el motor core, la API, la persistencia y la automatización. Esta separación permite que el motor de normalización pueda ser reutilizado desde Streamlit, FastAPI, CLI, extensión de VS Code o futuros agentes IA. El componente de persistencia queda desacoplado para permitir que el análisis local funcione aun cuando Supabase no esté disponible.



Fuente: elaboración propia basada en el análisis del repositorio DataQuest, FD03/SRS y FD05.

Figura 5. Arquitectura técnica ampliada de DataQuest. Fuente: elaboración propia.

11. ACTORES CLAVES

Actor / cargo	Responsabilidad	Participación	Entregable asociado
Estudiante desarrollador	Construye, integra, documenta y sustenta el proyecto.	Alta	Código, pruebas, FD06 y demo.
Docente evaluador	Valida alcance, coherencia académica y calidad del entregable.	Alta	Observaciones y aprobación.

DataQuest - FD06 / Propuesta de Proyecto

Usuario estudiante	Usa la plataforma para validar esquemas y aprender normalización.	Media	Feedback de uso y capturas.
Usuario docente	Puede usar reportes como apoyo en revisión preliminar.	Media	Criterios de evaluación.
Administrador técnico	Configura entorno, variables y despliegue.	Media	Instalación y configuración.
API consumer	Consume endpoint de normalización desde otro cliente.	Media	Prueba Postman/curl.
QA tester	Ejecuta pruebas funcionales, API, regresión y seguridad básica.	Alta	Matriz QA y evidencias.
Revisor académico	Evalúa consistencia de documentos FD03, FD05 y FD06.	Media	Checklist de documentación.

12. PAPEL Y RESPONSABILIDADES DEL PERSONAL

Actividad	Responsable	RACI	Responsabilidad	Evidencia
Análisis funcional	Estudiante desarrollador	R/A	Levantar problema, alcance, beneficios y restricciones.	FD06 revisado.
Desarrollo core	Desarrollador Python	R	Implementar parser, validadores, diagnóstico y sugerencias.	Pruebas unitarias.
Desarrollo UI	Desarrollador Streamlit	R	Construir flujo de entrada y visualización de resultados.	Capturas UI.
Desarrollo API	Backend developer	R	Exponer endpoint, validar entradas y errores.	Prueba Postman.
Base de datos	Administrador técnico	R/C	Configurar Supabase, RLS e historial.	Script modelos.sql.
QA	Analista QA	R/A	Diseñar y ejecutar matriz de pruebas.	Reporte QA.
Documentación	Redactor técnico	R	Actualizar FD03, FD05, FD06, manual y anexos.	Documentos finales.
Sustentación	Estudiante / expositor	A	Preparar demo, discurso y evidencias.	Presentación oral.
Mantenimiento	Equipo técnico	C/I	Corregir errores y ampliar funcionalidades.	Backlog de mejoras.

13. PLAN DE MONITOREO Y EVALUACIÓN

Indicador	Fórmula	Meta	Frecuencia	Responsable	Evidencia
Porcentaje de RF implementados	RF implementados / RF totales x 100	>= 75% para entrega académica	Semanal	Estudiante	Matriz RF
Pruebas ejecutadas	Casos ejecutados / casos planificados x 100	>= 80%	Por sprint	QA	Reporte Pytest/Postman
Pruebas aprobadas	Casos aprobados / casos ejecutados x 100	>= 85%	Por sprint	QA	Matriz QA
Tiempo promedio de validación	Promedio de segundos por esquema	< 3 s en esquemas académicos	Quincenal	Desarrollador	Medición local
Errores controlados	Errores con mensaje claro / errores totales	>= 90%	Quincenal	Backend	Logs/API
Cobertura documental	Secciones completas / secciones esperadas	100%	Cierre	Documentador	Checklist
Evidencias generadas	Capturas y reportes disponibles	>= 10 evidencias	Cierre	Estudiante	Anexo 05
Satisfacción académica	Encuesta breve 1 a 5	>= 4	Final	Docente/usuarios	Encuesta

14. CRONOGRAMA DEL PROYECTO

Actividad	Mes 1	Mes 2	Mes 3	Mes 4	Responsable	Entregable	Estado
Análisis y levantamiento	X				Estudiante	Alcance y problema	Planificado
FD03/SRS y arquitectura	X	X			Estudiante	Requerimientos y diagramas	Realizado/ajustable
Motor parser SQL		X			Desarrollador	Parser funcional	Implementado/mejorable
Validadores 1FN/2FN/3FN		X	X		Desarrollador	Motor core	Implementado/mejorable
Interfaz Streamlit		X	X		UI	Pantallas funcionales	Implementado/parcial
API FastAPI			X		Backend	Endpoint probado	Implementado
Supabase/PostgreSQL			X		BD	Historial y RLS	Planificado/configurable
VS Code Extension			X		Integración	Extensión funcional	Implementado técnico

DataQuest - FD06 / Propuesta de Proyecto

Docker/CI			X		DevOps	Pipeline	Implementado técnico
Pruebas y correcciones			X	X	QA	Matriz QA	Parcial/recomendado
FD05/FD06 y anexos				X	Documentador	Documentos finales	En elaboración
Sustentación				X	Estudiante	Demo final	Planificado

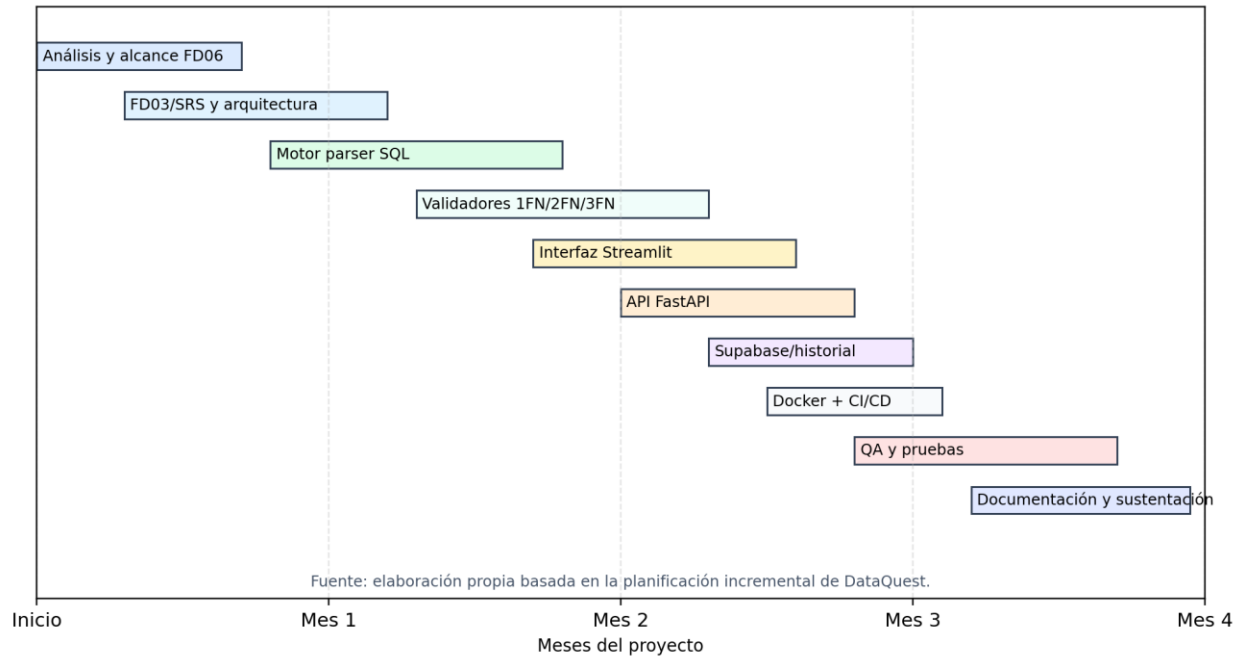


Figura 6. Cronograma tipo Gantt del proyecto DataQuest. Fuente: elaboración propia.

15. HITOS DE ENTREGABLES

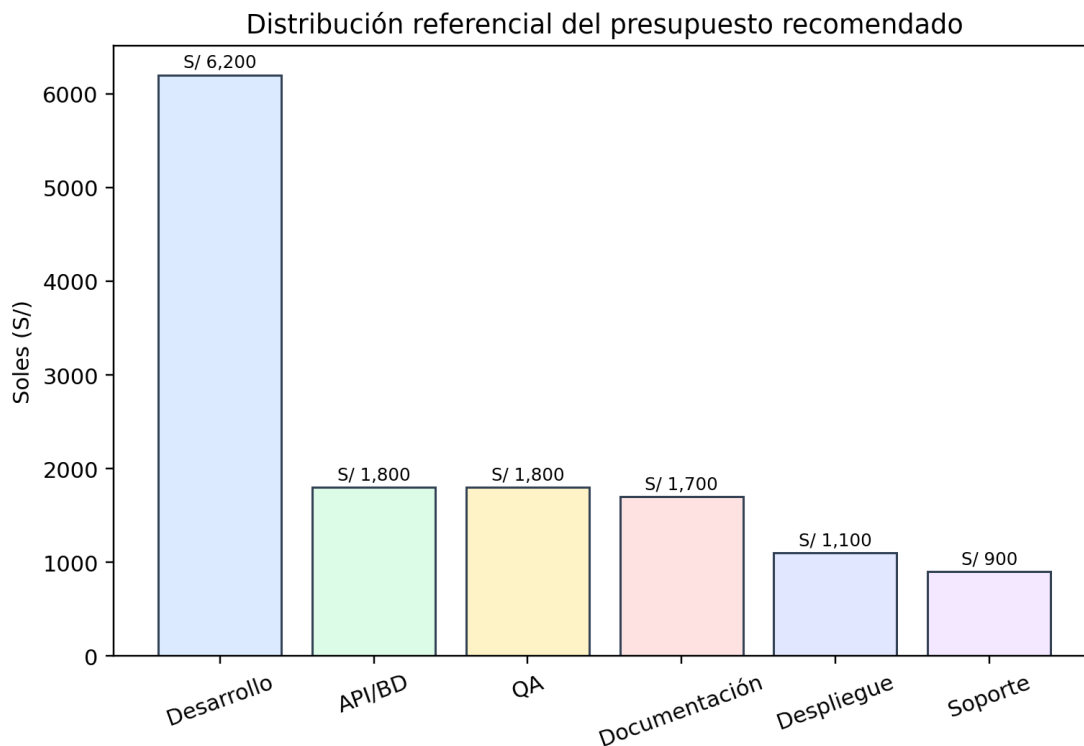
Hito	Fecha estimada	Entregable	Criterio de aceptación	Responsable
H1	Semana 1	Alcance aprobado	Problema, objetivo y alcance validados.	Estudiante
H2	Semana 3	Motor inicial	Parser detecta tablas y columnas.	Desarrollador
H3	Semana 5	Validadores 1FN/2FN/3FN	Casos base producen diagnóstico esperado.	Desarrollador
H4	Semana 7	Interfaz funcional	Usuario ejecuta flujo desde Streamlit.	UI
H5	Semana 9	API funcional	POST /api/normalizacion responde correctamente.	Backend
H6	Semana 11	Persistencia/historial	Supabase registra historial con RLS.	BD
H7	Semana 13	Pruebas QA	Matriz QA ejecutada con evidencias.	QA
H8	Semana 15	Documentación final	FD06 y anexos completos.	Documentador
H9	Semana 16	Sustentación	Demo y exposición listas.	Estudiante

PRESUPUESTO

16. PLANTEAMIENTO DE APLICACIÓN DEL PRESUPUESTO

El presupuesto de DataQuest se plantea como una inversión académica referencial. Su finalidad es estimar el esfuerzo técnico y documental requerido para convertir el proyecto en una solución sustentable, probada y defendible. No representa una propuesta comercial oficial; los montos se expresan en soles y se basan en horas estimadas de desarrollo, documentación, pruebas y soporte de sustentación.

- Desarrollo del motor core: parser, diagnóstico, validadores y sugerencias.
- Desarrollo de interfaz Streamlit y mejora de experiencia de usuario.
- Desarrollo y validación de API FastAPI.
- Configuración de persistencia con Supabase/PostgreSQL.
- Pruebas unitarias, API, interfaz, regresión y seguridad básica.
- Documentación FD06, diagramas, manuales y anexo único consolidado.
- Despliegue controlado con Docker y automatización CI/CD.
- Soporte de sustentación, evidencias y correcciones finales.



Fuente: elaboración propia basada en presupuesto académico referencial de DataQuest.

Figura 7. Distribución referencial del presupuesto recomendado. Fuente: elaboración propia.

17. PRESUPUESTO

Concepto	Cantidad	Costo unitario	Subtotal	Observación
Analista funcional / documentación	1	S/ 1,200.00	S/ 1,200.00	Levantamiento, alcance, FD06 y trazabilidad.
Desarrollador Python core	1	S/ 2,800.00	S/ 2,800.00	Parser, diagnóstico, validadores y sugerencias.
Desarrollador Streamlit UI	1	S/ 1,600.00	S/ 1,600.00	Pantallas, flujo de usuario y visualización.
Desarrollador API FastAPI	1	S/ 1,400.00	S/ 1,400.00	Endpoint, manejo de errores y documentación OpenAPI.
Base de datos Supabase/PostgreSQL	1	S/ 1,000.00	S/ 1,000.00	Historial, RLS y conexión.

DataQuest - FD06 / Propuesta de Proyecto

QA tester	1	S/ 1,800.00	S/ 1,800.00	Matriz de pruebas, regresión y evidencias.
Diseño de diagramas y evidencias	1	S/ 900.00	S/ 900.00	Diagramas, capturas y anexos.
DevOps académico	1	S/ 1,100.00	S/ 1,100.00	Docker, workflows y verificación de despliegue.
Capacitación / sustentación	1	S/ 700.00	S/ 700.00	Guion de demo y preparación de exposición.
Contingencia técnica	1	S/ 1,000.00	S/ 1,000.00	Correcciones no previstas.
TOTAL RECOMENDADO			S/ 13,500.00	Presupuesto académico referencial.

Escenarios de presupuesto

Escenario	Monto	Incluye	Uso recomendado
Mínimo	S/ 8,500.00	Documento, motor básico, Streamlit y API básica.	Adecuado para entrega académica reducida.
Recomendado	S/ 13,500.00	Motor, UI, API, Supabase, QA, diagramas y documentación completa.	Mejor equilibrio entre calidad y costo.
Extendido	S/ 18,500.00	Incluye mejora avanzada de VS Code Extension, Swagger detallado, mayor cobertura QA y despliegue cloud.	Recomendado para producto demostrable más sólido.

18. ANÁLISIS DE FACTIBILIDAD

Tipo	Evaluación	Argumento	Riesgo	Mitigación
Técnica	Alta	Las tecnologías del proyecto son accesibles y ya están presentes en el repositorio: Python, Streamlit, FastAPI, Supabase, Docker, GitHub Actions y VS Code Extension.	Dependencias o entorno mal configurado.	Documentar instalación y usar .env.example.
Económica	Viable	El proyecto usa herramientas libres o de bajo costo; el mayor costo corresponde a esfuerzo humano.	Sobrecosto por ampliación de alcance.	Controlar backlog y priorizar MVP.
Operativa	Alta	Estudiantes y docentes pueden usar la interfaz web sin aprender herramientas complejas.	Curva de aprendizaje inicial.	Manual breve y ejemplos SQL.
Académica	Alta	Atiende una necesidad clara de cursos de bases de datos y mejora trazabilidad de correcciones.	Dependencia excesiva de la herramienta.	Aclarar que es apoyo, no reemplazo docente.
Legal	Alta	No requiere datos personales reales si se usa en modo académico.	Credenciales expuestas.	Uso de variables de entorno y revisión de secretos.
Ambiental	Alta	Reduce impresión de guías y correcciones mediante reportes digitales.	Uso de recursos cloud innecesarios.	Usar entorno local para prácticas.
Mantenimiento	Media-alta	La arquitectura modular permite extender validadores y pruebas.	Pruebas insuficientes.	Ampliar Pytest y casos de regresión.

19. EVALUACIÓN FINANCIERA

La evaluación financiera de DataQuest se presenta como una estimación académica referencial. Dado que el proyecto no se plantea como producto comercial inmediato, el retorno se mide por ahorro de tiempo, reutilización en cursos, mejora de calidad académica y reducción de reprocesos. Para fines de propuesta, se consideran tres escenarios de valor.

Escenario	Inversión	Beneficio esperado	Retorno estimado	Relación beneficio/costo
Conservador	S/ 8,500.00	Uso en un curso y 25 estudiantes. Ahorro de 15 minutos por revisión preliminar.	Retorno académico en un ciclo.	B/C cualitativo positivo.
Moderado	S/ 13,500.00	Uso en varios cursos, 60 estudiantes y reutilización de reportes.	Retorno académico en 2 ciclos.	B/C referencial 1.25.
Optimista	S/ 18,500.00	Uso institucional, API, VS Code Extension y mayor cobertura QA.	Retorno académico y técnico en 2 a 3 ciclos.	B/C referencial 1.45.

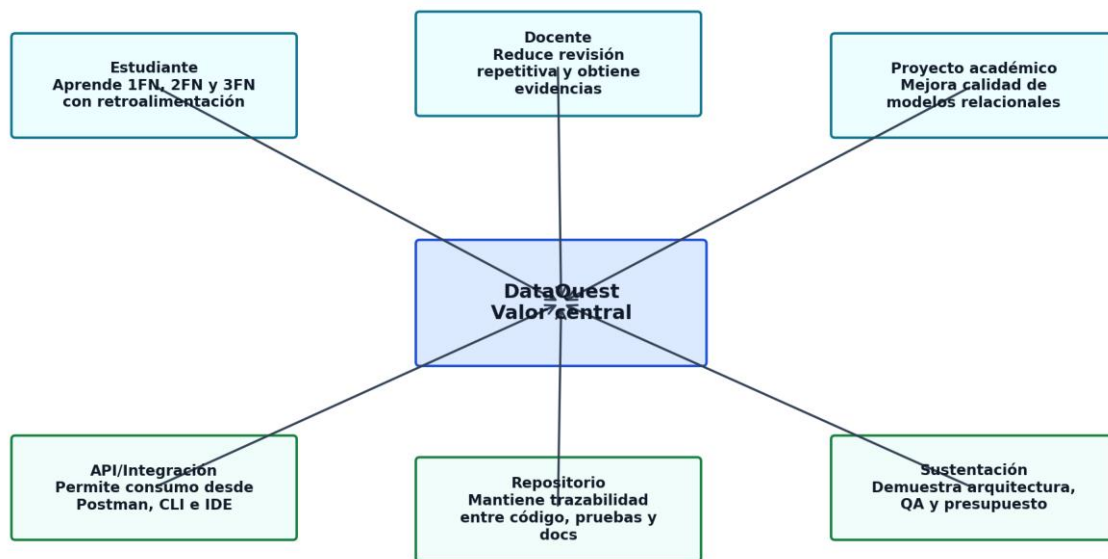
Supuestos de evaluación

- El valor económico se interpreta como ahorro de tiempo y mejora de calidad académica, no como ingreso comercial directo.
- La herramienta puede reutilizarse en múltiples prácticas, cursos y semestres.
- El costo de infraestructura se mantiene bajo usando entornos gratuitos o locales.
- El retorno aumenta si el sistema se usa como apoyo docente o repositorio de evidencias.

20. Resumen técnico ejecutivo

DataQuest se propone como una solución académica de validación de normalización de bases de datos relacionales orientada a estudiantes, docentes y desarrolladores principiantes. A diferencia de una revisión manual tradicional, la propuesta busca que el usuario ingrese un esquema SQL, reciba un diagnóstico por forma normal y obtenga sugerencias comprensibles para mejorar la estructura del modelo.

El enfoque del proyecto no consiste únicamente en detectar errores, sino en convertir el proceso de normalización en una experiencia formativa. Por ello, la propuesta combina interfaz web, motor lógico, API, repositorio documentado, pruebas y evidencias. El valor principal es pedagógico: reducir la brecha entre teoría de normalización y aplicación práctica en sentencias CREATE TABLE.



Fuente: elaboración propia basada en el análisis funcional, académico y técnico del proyecto DataQuest.

Figura 8. Mapa de valor académico y técnico de DataQuest. Fuente: elaboración propia.

20.1 Valor diferencial de DataQuest

Dimensión	Situación tradicional	Aporte de DataQuest	Evidencia esperada
Aprendizaje	El estudiante revisa reglas de 1FN, 2FN y 3FN de forma teórica.	Relaciona teoría con esquemas SQL reales, diagnóstico y sugerencias.	Capturas de validaciones y reportes.
Tiempo	La revisión depende de lectura manual y experiencia del evaluador.	Automatiza diagnóstico preliminar y permite repetir pruebas.	Comparación de tiempo antes/después.
Trazabilidad	Correcciones suelen quedar en comentarios sueltos o apuntes.	Genera evidencias, historial o salidas JSON reutilizables.	Historial, reportes o resultados exportados.
Integración	La validación ocurre fuera del flujo de desarrollo.	Permite uso desde Streamlit, API, CLI o VS Code si está disponible.	Prueba de API, CLI y extensión.
Calidad	El proyecto académico puede no demostrar QA formal.	Incluye pruebas unitarias, API, UI, riesgos y criterios de aceptación.	Plan QA y ejecución de casos.

20.2 Principios rectores de la propuesta

- Claridad pedagógica: cada diagnóstico debe explicar por qué existe una posible infracción de normalización.
- Transparencia técnica: toda limitación del motor debe ser reconocida, especialmente cuando se usen heurísticas.
- Trazabilidad académica: cada resultado debe poder vincularse con un requerimiento, una prueba y una evidencia.
- Arquitectura modular: interfaz, core, API, persistencia, pruebas y documentación deben poder evolucionar sin acoplamiento excesivo.
- Seguridad responsable: no exponer credenciales, proteger variables de entorno y restringir CORS en despliegue real.
- Evolución incremental: priorizar un producto funcional y demostrable antes que prometer automatizaciones no verificadas.

21. Diagnóstico profundo del proyecto DataQuest

El repositorio DataQuest evidencia una estructura modular con archivos principales app.py, api.py, carpetas core, controllers, db, views, visualizacion, tests, terraform y vscode-extension. Esta organización favorece una propuesta sólida porque permite separar vista, controladores, lógica de normalización, persistencia, visualización, automatización y pruebas.

El README del proyecto declara un estado en desarrollo y presenta componentes de valor como Streamlit, PostgreSQL/Supabase, VS Code Extension, Agent Skill, CLI, validadores 1FN/2FN/3FN, Graphviz/NetworkX, historial y pruebas. En el FD06 enriquecido se debe tratar esta información con criterio: no todo lo mencionado debe asumirse como completamente productivo si existen archivos con pruebas dummy o pasos pendientes.

21.1 Inventario técnico real del repositorio

Elemento	Ubicación/archivo	Función	Estado documentable
Interfaz web	app.py y views/	Presentar la experiencia Streamlit, pantallas y resultados.	Implementado/parcial según ejecución real.
Motor core	core/parser.py, diagnostico.py, validador_1fn.py, validador_2fn.py, validador_3fn.py	Analizar SQL y aplicar reglas de normalización.	Implementado con lógica heurística.
API	api.py	Exponer POST /api/normalizacion y raíz de estado.	Implementado, con mejora requerida en CORS y errores.
CLI	core/cli.py	Ejecutar análisis por archivo o stdín y devolver JSON.	Implementado.
Persistencia	db/conexion.py, db/auth.py, db/modelos.sql	Soporte para Supabase/PostgreSQL, usuarios e historial.	Parcial/depende de configuración.
Visualización	visualizacion/diagrama_er.py, diagrama_schema.py	Generar diagramas ER o schema.	Implementado/parcial según dependencias.
Extensión VS Code	vscode-extension/	Analizar desde editor e insertar reporte.	Declarado como completado en README; validar en ejecución.
Tests	tests/unit, integration, ui, bdd	Pruebas de núcleo, integración, UI y BDD.	Parcial: existen tests dummy/pass que deben completarse.
Infraestructura	Dockerfile, github, terraform/	Contenerización, CI/CD e IaC.	Presente, requiere validación.
Agent Skill	.agents/skills/validador_normalizacion/SKILL.md	Documentar lógica para uso con agentes IA.	Presente según estructura del repositorio.

21.2 Hallazgos QA relevantes

Código	Hallazgo	Impacto	Recomendación
QA-01	La API permite CORS con allow_origins=["***"].	Adecuado para demo, riesgoso en producción.	Restringir dominios permitidos antes de despliegue real.
QA-02	Algunas pruebas contienen pass o assert True.	La cobertura real puede ser baja aunque exista carpeta tests.	Reemplazar dummies por casos reales de parser, 1FN, 2FN, 3FN y API.
QA-03	El diagnóstico de dependencias puede usar heurísticas.	Puede producir falsos positivos o falsos negativos.	Permitir ingreso manual de dependencias funcionales como mejora.
QA-04	El historial depende de Supabase y variables .env.	El sistema puede fallar si no hay configuración externa.	Agregar modo local/mock y validación de configuración.
QA-05	Los errores internos devuelven mensaje con detalle de excepción.	Puede exponer detalles técnicos.	Estandarizar errores y registrar trazas solo en servidor.
QA-06	La extensión VS Code requiere validación funcional en entorno Node.	Puede no demostrarse si no se instala y prueba.	Incluir evidencia de npm install, F5 y comando ejecutado.

21.3 Decisiones técnicas que deben mantenerse

La propuesta debe mantener Python como núcleo por su facilidad para procesar texto, aplicar reglas y generar diagnósticos. Streamlit es adecuado para una demostración académica rápida y visual. FastAPI permite demostrar integración HTTP/JSON y documentación OpenAPI. Supabase/PostgreSQL ofrece una base de persistencia viable para historial y autenticación, pero debe usarse sin exponer credenciales. Docker, GitHub Actions y Terraform fortalecen la dimensión profesional siempre que se documenten como capacidades verificables y no solo decorativas.

22. Análisis estratégico y propuesta de valor ampliada

22.1 Población objetivo segmentada

Segmento	Necesidad principal	Dolor actual	Respuesta de DataQuest
Estudiantes iniciales de BD	Comprender normalización con ejemplos.	Confunden claves, atributos atómicos y dependencias.	Diagnóstico guiado por 1FN, 2FN y 3FN.
Estudiantes con proyectos	Validar su modelo antes de sustentación.	No tienen evidencia técnica de corrección.	Reporte y sugerencias trazables.
Docentes	Revisar muchos modelos de forma preliminar.	La revisión repetitiva consume tiempo.	Herramienta de apoyo, no reemplazo del juicio docente.
Desarrolladores principiantes	Detectar redundancia en diseños SQL.	Usan tablas grandes o mal separadas.	Recomendaciones sobre descomposición y claves.
Evaluador académico	Ver coherencia entre código, documentos y pruebas.	Proyectos suelen carecer de QA y trazabilidad.	FD06 integra alcance, presupuesto, riesgos y QA.

22.2 Propuesta de valor resumida

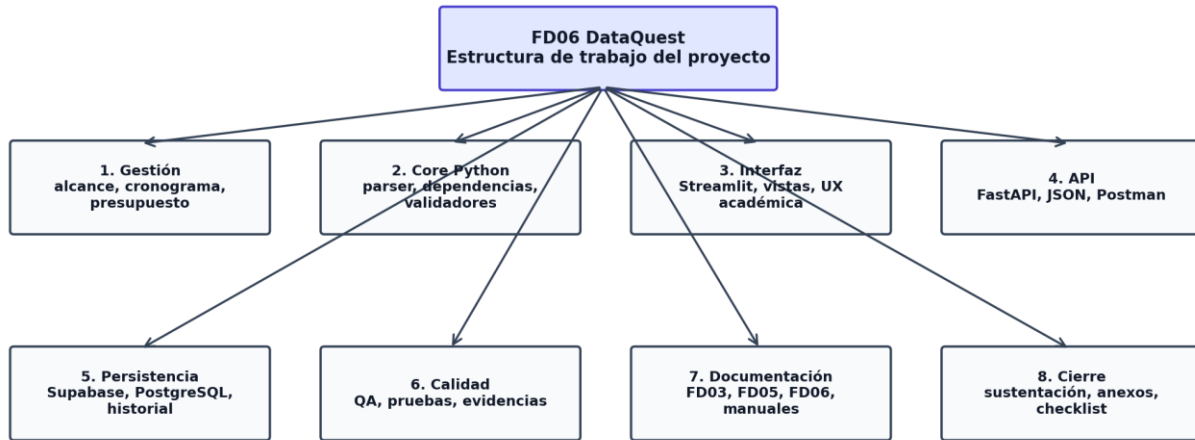
DataQuest ofrece una propuesta de valor basada en cuatro elementos: aprendizaje guiado, diagnóstico automatizado, evidencia documentable e integración técnica. Esto significa que el estudiante no solo obtiene un resultado, sino una explicación que puede usar para corregir su esquema y sustentar su decisión frente al docente.

Eje de valor	Descripción	Indicador de éxito
Aprendizaje	El sistema explica hallazgos de normalización en lenguaje comprensible.	80% de usuarios piloto entiende la causa del diagnóstico.
Velocidad	El análisis inicial de un esquema se realiza en segundos o pocos minutos.	Reducción de al menos 50% del tiempo preliminar de revisión.
Calidad	El resultado se evalúa con pruebas y criterios de aceptación.	90% de pruebas críticas aprobadas.
Trazabilidad	Cada validación puede registrarse, exportarse o evidenciarse.	Evidencia disponible por cada caso de prueba.
Escalabilidad académica	El sistema puede ser usado por múltiples estudiantes en prácticas.	Reutilización en más de una actividad académica.

22.3 Alineamiento con objetivos académicos

- Apoya el logro de competencias en modelamiento lógico de bases de datos.
- Facilita la identificación de anomalías de inserción, actualización y eliminación.
- Promueve buenas prácticas de diseño antes de crear bases de datos físicas.
- Relaciona teoría de normalización con sintaxis SQL real.
- Permite convertir la revisión de esquemas en un proceso repetible y documentado.
- Refuerza habilidades de programación, pruebas, API, documentación y DevOps.

23. EDT/WBS, backlog e historias de usuario



Fuente: elaboración propia basada en EDT/WBS de la propuesta DataQuest.

Figura 9. Estructura de desglose del trabajo para DataQuest. Fuente: elaboración propia.

23.1 Backlog de producto priorizado

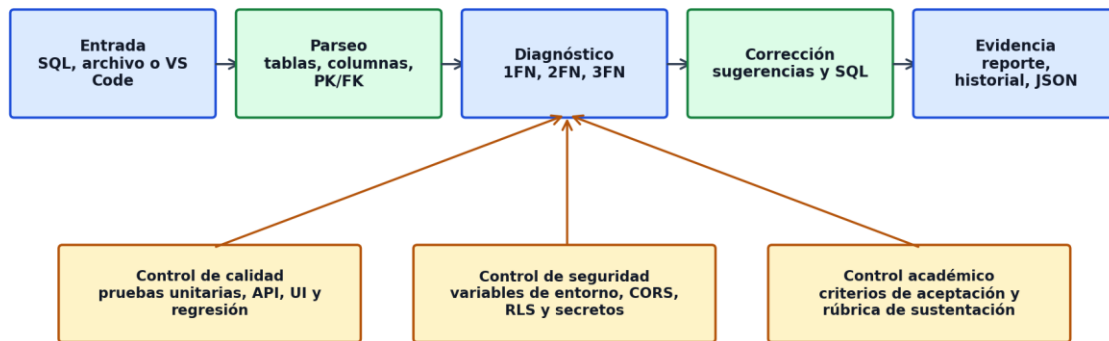
ID	Épica	Historia de usuario	Prioridad	Entrega
BP-01	Ingreso de SQL	Como estudiante quiero pegar un CREATE TABLE para iniciar la validación.	Alta	MVP
BP-02	Parser SQL	Como sistema quiero extraer tablas, campos, PK y FK para analizar el esquema.	Alta	MVP
BP-03	Validador 1FN	Como usuario quiero saber si existen atributos no atómicos o repetitivos.	Alta	MVP
BP-04	Validador 2FN	Como usuario quiero detectar dependencias parciales cuando existan claves compuestas.	Alta	MVP
BP-05	Validador 3FN	Como usuario quiero identificar posibles dependencias transitivas.	Alta	MVP
BP-06	Sugerencias	Como estudiante quiero recibir recomendaciones concretas de corrección.	Alta	MVP
BP-07	Reporte visual	Como usuario quiero ver resultados ordenados por nivel de normalización.	Alta	MVP
BP-08	API	Como integrador quiero consumir el motor desde POST /api/normalizacion.	Alta	MVP
BP-09	Historial	Como usuario quiero guardar validaciones para comparar avances.	Media	Incremento 2
BP-10	Exportación	Como usuario quiero descargar un reporte o SQL sugerido.	Media	Incremento 2
BP-11	VS Code	Como desarrollador quiero validar desde mi editor.	Media	Incremento 3
BP-12	QA automatizado	Como equipo quiero pruebas reales que respalden la calidad.	Alta	MVP
BP-13	Seguridad	Como responsable quiero evitar exposición de secretos y CORS abierto.	Alta	MVP
BP-14	Documentación	Como evaluador quiero manuales, evidencias y trazabilidad.	Alta	MVP

23.2 Historias de usuario detalladas con criterios de aceptación

ID	Nombre	Criterio de aceptación	Condición	Prioridad
HU-01	Validar SQL correcto	Dado un CREATE TABLE válido, cuando el usuario solicita análisis, entonces el sistema muestra tablas detectadas, nivel y violaciones.	SQL válido con al menos una tabla.	Alta
HU-02	Controlar entrada vacía	Dado un campo SQL vacío, cuando se llama a la API, entonces devuelve success=false y error 400 controlado.	No debe lanzar traceback al usuario.	Alta
HU-03	Detectar tabla sin PK	Dado un esquema sin clave primaria, entonces el diagnóstico advierte riesgo de identificación de filas.	Mensaje claro en resultados.	Alta
HU-04	Detectar atributo multivaluado	Dado un campo telefonos o lista en una tabla, entonces sugiere separarlo en tabla relacionada.	Explica relación con 1FN.	Alta
HU-05	Detectar dependencia parcial	Dado una clave compuesta, entonces identifica atributos que dependen de parte de la clave cuando aplique.	Explica 2FN y sugiere descomposición.	Alta
HU-06	Detectar dependencia transitiva	Dado un atributo no clave que determina otro no clave, entonces marca posible 3FN.	Sugiere crear entidad relacionada.	Alta
HU-07	Generar reporte JSON	Dado un análisis por API, entonces devuelve nivel_actual, violaciones y resumen.	Contrato JSON estable.	Alta
HU-08	Guardar historial	Dado un usuario configurado, entonces se registra la validación en PostgreSQL/Supabase.	Aplica solo si variables .env están configuradas.	Media
HU-09	Exportar sugerencias	Dado un diagnóstico, entonces el usuario puede obtener recomendaciones en formato descargable si el módulo existe.	No debe prometer PDF si no se valida.	Media
HU-10	Validar desde VS Code	Dado un archivo SQL abierto, entonces la extensión inserta reporte al final del documento.	Evidencia con comando y captura.	Media
HU-11	Ejecutar CLI	Dado un archivo SQL, cuando se ejecuta core/cli.py, entonces se imprime JSON estructurado.	Código de salida controlado.	Alta

HU-12	Visualizar diagrama	Dado un esquema parseado, entonces se genera vista ER o schema si dependencias están instaladas.	Captura o archivo de diagrama.	Media
HU-13	Controlar SQL inválido	Dado un SQL mal formado, entonces el sistema muestra error comprensible.	Sin caída total de la interfaz.	Alta
HU-14	Pruebas automatizadas	Dado el repositorio, cuando se ejecuta pytest, entonces las pruebas críticas no son dummy.	Reemplazar pass por asserts reales.	Alta
HU-15	Sustentación docente	Dado el FD06 final, entonces existe coherencia entre problema, objetivos, solución, costos, riesgos y QA.	Checklist jurado completo.	Alta

24. Arquitectura técnica, ciclo de datos, API y seguridad



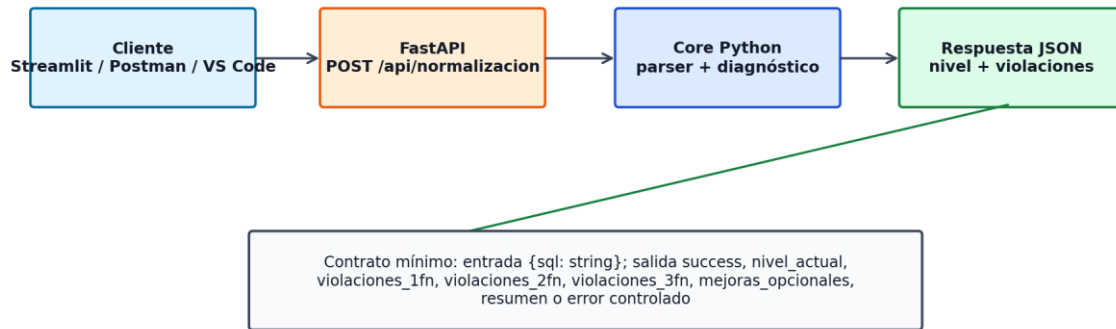
Fuente: elaboración propia basada en el ciclo de procesamiento de DataQuest.

Figura 10. Ciclo de datos y evidencia del flujo de validación. Fuente: elaboración propia.

24.1 Arquitectura lógica enriquecida

La arquitectura se interpreta como una variante MVC extendida: las vistas corresponden a Streamlit y potencialmente VS Code; los controladores se ubican en controllers/ y CLI; el modelo lógico vive en core/; la persistencia y autenticación residen en db/; la API externa se expone con FastAPI. Esta separación facilita pruebas, mantenimiento y ampliación futura.

Capa	Responsabilidad	Módulos relacionados	Riesgo principal	Mitigación
Presentación	Capturar SQL, mostrar diagnóstico y evidencias.	app.py, views/	Interfaz poco clara para principiantes.	Mensajes pedagógicos, ejemplos y validaciones visibles.
Controladores	Coordinar flujo entre UI, core y datos.	controllers/, core/cli.py	Acoplamiento con lógica.	Mantener controladores delgados.
Core	Parsear, diagnosticar y sugerir mejoras.	core/parser.py, dependencias.py, validador *.py	Heurísticas incompletas.	Permitir ingreso manual de dependencias.
API	Exponer servicio HTTP/JSON.	api.py	CORS abierto o errores expuestos.	Restringir CORS y estandarizar errores.
Datos	Guardar usuarios, historial y resultados.	db/, modelos.sql	Configuración externa faltante.	Variables .env y modo local.
DevOps	Construir, probar y desplegar.	Dockerfile, .github, terraform/	Scripts no validados.	Pipeline con pruebas reales.



Fuente: elaboración propia basada en api.py y la especificación de integración de DataQuest.

Figura 11. Contrato conceptual de la API de normalización. Fuente: elaboración propia.

24.2 Especificación ampliada del endpoint principal

Elemento	Detalle
Método y ruta	POST /api/normalizacion
Propósito	Recibir sentencias SQL CREATE TABLE y devolver diagnóstico de normalización.
Entrada esperada	{ "sql": "CREATE TABLE ..." }
Salida exitosa	success=true, nivel_actual, violaciones_1fn, violaciones_2fn, violaciones_3fn, mejoras_opcionales, resumen.
Error por entrada vacía	success=false, error.message="El campo sql no puede estar vacío", code=400.
Error por esquema no detectado	success=false, mensaje sobre tablas válidas no encontradas.
Error interno	Debe devolver mensaje controlado sin exponer detalles sensibles en versión final.
Prueba recomendada	Postman/curl con SQL normalizado, SQL con 1FN violada, SQL con dependencia parcial, SQL vacío y JSON inválido.
Mejora prioritaria	Agregar autenticación opcional, limitación de tamaño de payload y CORS restringido para producción.

24.3 Seguridad y privacidad

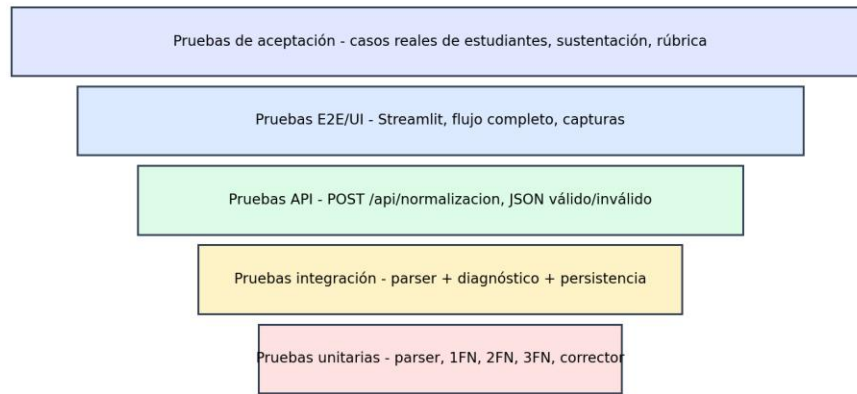
Riesgo	Control mínimo	Control recomendado	Evidencia
Credenciales expuestas	Uso de .env y .env.example.	Secret scanning en CI y rotación de claves.	Archivo .env no versionado.
CORS abierto	Permitido solo para demo local.	Lista blanca de orígenes confiables.	Configuración documentada en api.py.
Datos de esquemas privados	No compartir historial entre usuarios.	RLS en Supabase/PostgreSQL.	Prueba de acceso cruzado.
Errores internos	Mensaje controlado al usuario.	Logs separados del output público.	Captura de error controlado.
Uso abusivo de API	Validar tamaño de entrada.	Rate limiting y timeout.	Prueba con payload grande.

24.4 Plan DevOps e infraestructura

La presencia de Dockerfile, GitHub Actions y Terraform permite proponer un flujo DevOps académico. Sin embargo, el FD06 debe exigir evidencia: construcción de imagen, ejecución local, pruebas automatizadas y documentación de variables. No se debe afirmar despliegue productivo si solo existe configuración inicial.

Fase	Actividad	Herramienta	Criterio de aceptación
Build	Construir entorno reproducible.	Dockerfile	Imagen construye sin errores y ejecuta la app.
Test	Ejecutar pruebas automáticas.	pytest, Playwright, Postman	Pruebas críticas con asserts reales.
Security	Detectar secretos y dependencias inseguras.	Snyk/semgrep/secret scan	Reporte sin secretos expuestos.
Deploy demo	Ejecutar Streamlit y API en entorno local o nube.	Streamlit, uvicorn, Docker	URLs accesibles y evidencias.
IaC	Documentar infraestructura como código.	Terraform	Plan documentado, sin credenciales en variables.

25. Plan QA extendido y criterios de aceptación



Estrategia: iniciar con pruebas unitarias reales, reemplazar dummies/pass, automatizar regresión y documentar evidencia.

Fuente: elaboración propia basada en el plan QA recomendado para DataQuest.

Figura 12. Estrategia de pruebas por niveles para DataQuest. Fuente: elaboración propia.

25.1 Objetivo del plan QA

El plan de calidad tiene como objetivo demostrar que DataQuest no solo funciona en casos favorables, sino que responde de forma controlada ante errores, entradas ambiguas, esquemas mal diseñados y configuraciones incompletas. El proyecto debe evolucionar desde pruebas dummy hacia pruebas reales con entradas SQL representativas.

25.2 Matriz de pruebas funcionales principales

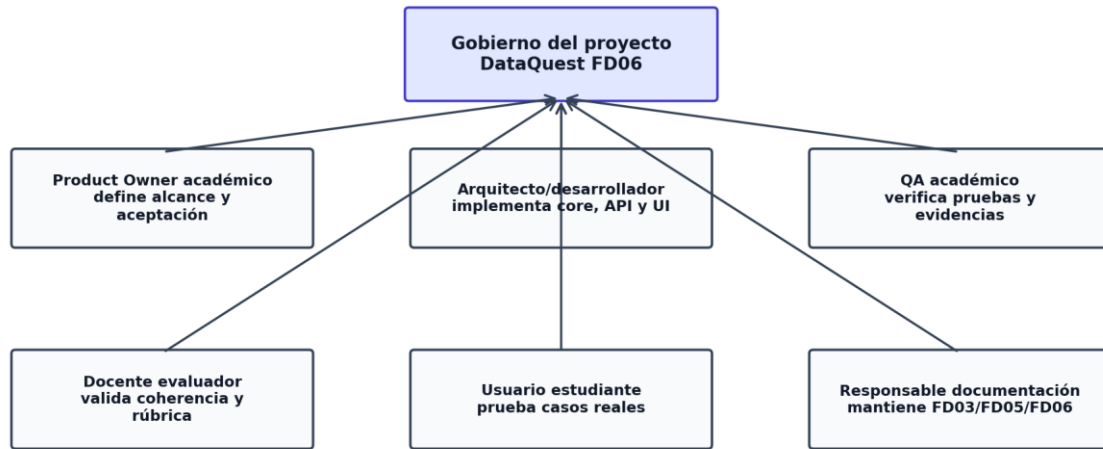
ID	Escenario	Entrada	Resultado esperado	Prioridad
TC-01	Entrada vacía	{"sql":""}	Error 400 controlado	Alta
TC-02	SQL sin CREATE TABLE	SELECT * FROM alumnos;	Mensaje: no se encontraron tablas válidas	Alta
TC-03	Tabla simple normalizada	CREATE TABLE alumno(id INT PRIMARY KEY, nombre VARCHAR(50));	Sin violaciones críticas	Alta
TC-04	Tabla sin PK	CREATE TABLE alumno(nombre VARCHAR(50));	Advertencia sobre identificación de filas	Alta
TC-05	Atributo multivaluado	telefonos VARCHAR(200)	Posible violación 1FN	Alta
TC-06	Clave compuesta con dependencia parcial	detalle(id_pedido,id_producto,nombre_producto)	Violación potencial 2FN	Alta
TC-07	Dependencia transitiva	alumno(id, id_carrera, nombre_carrera)	Violación potencial 3FN	Alta
TC-08	Tabla sábana	tabla con más de 6 atributos no clave	Sugerencia de partición	Media
TC-09	API JSON válido	POST con SQL correcto	success=true	Alta
TC-10	API JSON inválido	payload sin campo sql	Error de validación Pydantic	Alta
TC-11	CLI con archivo inexistente	core/cli.py nofile.sql	Error controlado y código salida 1	Media
TC-12	Historial sin Supabase	sin .env	No debe bloquear validación local	Media
TC-13	VS Code extension	archivo .sql abierto	Inserta bloque de reporte si está configurada	Media
TC-14	Carga de archivo grande	SQL con varias tablas	Tiempo aceptable y sin crash	Media
TC-15	Regresión core	suite pytest	Todas las pruebas críticas pasan	Alta

25.3 Criterios de aceptación globales

- El sistema debe analizar al menos un esquema SQL válido desde interfaz web, API y CLI.
- El endpoint POST /api/normalizacion debe responder con JSON estable para casos válidos e inválidos.
- El diagnóstico debe separar hallazgos de 1FN, 2FN y 3FN.
- Las sugerencias deben ser comprensibles y accionables para un estudiante.
- Los errores no deben mostrar trazas internas al usuario final en modo de entrega.
- La documentación debe explicar limitaciones del enfoque heurístico.
- Los tests dummy/pass deben ser reemplazados por pruebas reales antes de la sustentación final.

- El documento FD06 debe mantener consistencia con FD03, FD05, repositorio y evidencias.

26. Gobierno, monitoreo, comunicación y adopción



Fuente: elaboración propia basada en la gobernanza académica propuesta para DataQuest.

Figura 13. Modelo de gobierno académico para DataQuest. Fuente: elaboración propia.

26.1 Matriz RACI extendida

Actividad	Estudiante desarrollador	Docente	QA/Revisor	Usuario piloto	Evidencia
Definir alcance	R	A	C	C	Acta o sección de alcance
Diseñar arquitectura	R	C	C	I	Diagrama y explicación
Implementar core	R/A	I	C	I	Código core/
Implementar API	R/A	I	C	I	api.py y Postman
Pruebas funcionales	R	I	A	C	Matriz QA
Validación académica	C	A	R	C	Rúbrica y observaciones
Documentación FD06	R/A	C	C	I	Documento final
Sustentación	R	A	C	I	Presentación y demo

26.2 Plan de monitoreo y evaluación enriquecido



Fuente: elaboración propia basada en Balanced Scorecard académico de DataQuest.

Figura 14. Scorecard de monitoreo académico para DataQuest. Fuente: elaboración propia.

Indicador	Fórmula	Meta	Frecuencia	Responsable	Evidencia
Avance de requerimientos	RF implementados / RF planificados	>= 80% MVP	Semanal	Desarrollador	Backlog actualizado
Pruebas aprobadas	Pruebas OK / pruebas ejecutadas	>= 90% críticas	Por sprint	QA	Reporte pytest/Postman
Errores críticos abiertos	Cantidad de errores severidad alta	0 al cierre	Semanal	QA	Issue tracker
Tiempo de validación	Promedio por esquema	< 5 s demo local	Sprint 6+	Desarrollador	Log de ejecución
Cobertura documental	Secciones completas / secciones requeridas	100% FD06	Final	Documentador	DOCX final
Satisfacción piloto	Promedio encuesta 1-5	>= 4	Cierre	Docente/usuario	Formulario o acta
Trazabilidad	RF con prueba y evidencia / RF total	>= 85%	Final	QA	Matriz trazabilidad
Seguridad básica	Controles cumplidos / controles definidos	>= 90%	Final	Dev/QA	Checklist seguridad

26.3 Plan de comunicación

Audiencia	Mensaje clave	Canal	Frecuencia	Responsable
Docente	Estado, evidencias, riesgos y decisiones de alcance.	Documento, correo, sustentación	Por entrega	Estudiante
Usuario estudiante	Cómo ingresar SQL, interpretar resultados y corregir.	Manual y demo	Sprint final	Estudiante
QA/Revisor	Casos de prueba, defectos y evidencias.	Matriz QA	Semanal	QA
Equipo técnico	Cambios en core, API, persistencia y configuración.	README, issues	Por sprint	Desarrollador
Evaluador académico	Valor, viabilidad, presupuesto y alcance.	FD06 y exposición	Entrega final	Responsable proyecto

26.4 Plan de capacitación y adopción

La adopción de DataQuest se organiza en una demostración guiada. Primero se explica el problema académico; luego se muestra un esquema incorrecto, se ejecuta el diagnóstico, se interpreta el resultado y se corrige el modelo. Finalmente se enseña el uso de API o CLI para demostrar integración técnica.

Sesión	Tema	Duración	Resultado esperado
S1	Introducción a DataQuest y normalización aplicada	30 min	Usuario entiende propósito y límites.
S2	Uso de Streamlit y lectura de resultados	45 min	Usuario ejecuta validación completa.
S3	Casos de error y corrección de esquemas	45 min	Usuario interpreta sugerencias.
S4	API/CLI para integración técnica	30 min	Usuario prueba endpoint o CLI.
S5	Sustentación y evidencias	30 min	Usuario sabe presentar resultados.

27. Presupuesto enriquecido y evaluación financiera académica

27.1 Supuestos presupuestales

El presupuesto de DataQuest se plantea como referencial académico. No representa una propuesta comercial oficial, pero permite demostrar planificación realista. Se consideran costos de desarrollo, pruebas, documentación, diseño, despliegue y soporte. Como el proyecto usa tecnologías mayormente libres, el mayor costo está en horas de trabajo técnico y documentación.

Supuesto	Valor considerado	Justificación
Duración	4 meses	Periodo académico razonable para análisis, construcción, QA y entrega.
Equipo base	1 responsable principal con apoyo QA/documentación	Adecuado para proyecto universitario individual.
Infraestructura	Entorno local y servicios gratuitos o de bajo costo	Streamlit/FastAPI/Supabase pueden operar en demo.
Licencias	Preferentemente software libre	Python, FastAPI, Streamlit y PostgreSQL son accesibles.
Contingencia	10% a 15%	Cubre ajustes, pruebas adicionales y documentación final.



Fuente: elaboración propia basada en Balanced Scorecard académico de DataQuest.

Figura 15. Indicadores de valor usados para justificar la inversión académica. Fuente: elaboración propia.

27.2 Presupuesto por escenarios

Categoría	Mínimo S/	Recomendado S/	Extendido S/	Observación
Análisis y gestión	800	1,200	1,600	Alcance, cronograma, riesgos y seguimiento.
Desarrollo core Python	2,200	3,200	4,200	Parser, dependencias, validadores y corrector.
Interfaz Streamlit	1,000	1,600	2,200	Vistas, UX, mensajes y evidencias.
API FastAPI	900	1,400	2,000	Endpoint, validaciones, errores y docs.
Persistencia Supabase/PostgreSQL	700	1,100	1,600	Historial, modelos y configuración.
VS Code/CLI/Integración	600	1,000	1,600	Integración IDE y scripts.
QA y pruebas	1,000	1,800	2,600	Unit, API, UI, BDD y regresión.
Documentación	1,200	2,000	2,800	FD03, FD05, FD06, manuales y anexos.
Despliegue/DevOps	500	900	1,400	Docker, CI/CD, configuración.
Capacitación/sustentación	500	800	1,200	Demo, guion, entrenamiento.
Contingencia	900	1,500	2,300	Ajustes y mejoras finales.
TOTAL	10,300	16,500	23,500	Montos referenciales académicos.

27.3 Retorno académico esperado

El retorno de DataQuest no debe medirse únicamente como ingreso monetario, sino como valor académico: ahorro de tiempo de revisión, mejora de comprensión, reutilización en prácticas, reducción de errores de modelado y fortalecimiento de evidencias para sustentación. Aun así, puede estimarse un ahorro referencial de horas si el sistema reduce revisiones manuales repetitivas.

Escenario	Uso mensual estimado	Ahorro por validación	Valor hora S/	Beneficio mensual S/	Retorno estimado
-----------	----------------------	-----------------------	---------------	----------------------	------------------

DataQuest - FD06 / Propuesta de Proyecto

Conservador	20 validaciones	10 min	15	50	Valor principalmente académico.
Moderado	60 validaciones	15 min	15	225	Útil para cursos y prácticas.
Optimista	150 validaciones	20 min	20	1,000	Viable como herramienta institucional piloto.

27.4 Evaluación beneficio/costo referencial

Indicador	Cálculo referencial	Interpretación
Inversión recomendada	S/ 16,500	Incluye desarrollo, QA, documentación y contingencia.
Beneficio académico anual moderado	S/ 2,700 en ahorro de tiempo + valor formativo	El retorno monetario no captura todo el valor educativo.
Beneficio/costo directo	0.16 en primer año solo por ahorro de tiempo	No es un producto comercial puro; es inversión educativa.
Beneficio/costo ampliado	Mayor a 1 si se reutiliza en cursos, prácticas y portafolio	El valor aumenta con reutilización institucional.
Periodo de maduración	1 a 2 ciclos académicos	Tiempo razonable para mejora y adopción.

28. Riesgos, mitigaciones y trazabilidad ampliada

28.1 Matriz de riesgos extendida

ID	Riesgo	Prob.	Impacto	Nivel	Mitigación
R-01	El parser no interpreta todas las variantes SQL.	Media	Alta	Alto	Limitar sintaxis soportada y documentar ejemplos válidos.
R-02	Las dependencias funcionales no se infieren correctamente.	Alta	Alta	Crítico	Agregar módulo de ingreso manual de dependencias.
R-03	Pruebas dummy reducen la confianza del proyecto.	Alta	Media	Alto	Reemplazar pass/assert True por pruebas reales.
R-04	Supabase no está configurado en demo.	Media	Media	Medio	Preparar modo local o datos de ejemplo.
R-05	La API expone CORS abierto.	Media	Alta	Alto	Restringir allow_origins y documentar modo demo.
R-06	Credenciales en archivos o repositorio.	Baja	Alta	Alto	Usar .env, .gitignore y secret scanning.
R-07	El documento promete funcionalidades no verificadas.	Media	Alta	Alto	Clasificar implementado/parcial/planificado.
R-08	El usuario no entiende el diagnóstico.	Media	Media	Medio	Mensajes pedagógicos y ejemplos antes/después.
R-09	Extensión VS Code no se puede demostrar.	Media	Media	Medio	Preparar evidencia o documentar como pendiente.
R-10	Sobrecarga de alcance en 4 meses.	Media	Alta	Alto	Priorizar MVP: core + Streamlit + API + QA.
R-11	Errores internos se muestran al usuario.	Media	Media	Medio	Estandarizar mensajes y logs privados.
R-12	Falta de evidencia visual final.	Media	Media	Medio	Reservar sprint final para capturas y anexos.

28.2 Trazabilidad problema-objetivo-requerimiento-prueba

ID	Problema asociado	Objetivo	RF	Prueba/evidencia	Prioridad
TR-01	Validación manual lenta	OE1/OE5	RF-01/RF-09	TC-01, TC-03	Alta
TR-02	Desconocimiento de 1FN	OE2	RF-04	TC-05	Alta
TR-03	Dificultad con 2FN	OE3	RF-05	TC-06	Alta
TR-04	Dificultad con 3FN	OE4	RF-06	TC-07	Alta
TR-05	Falta de evidencia	OE6/OE10	RF-12/RF-19	TC-09, checklist	Alta
TR-06	Necesidad de integración	OE7	RF-10	TC-09, TC-10	Alta
TR-07	Riesgos de calidad	OE9	RF-15/RF-20	TC-15	Media
TR-08	Persistencia de resultados	OE8	RF-11	TC-12	Media